

* Assignment No. 2 *

Q.1 What is stack? Explain different applications of stack.

→ stack:

- A stack is a linear data structure that follows the principle of LIFO (Last In First out) the last element inserted is the first one to be removed.
- It is similar to a stack of plates where you can only add or remove the plate from the top.

• Basic operations of stack:

1] Push:

Insert an element into the stack.

2] Pop:

Remove an element from the stack.

3] Peek/Top:

view the top element.

4] is Empty:

check if stack is empty

5] is Full:

check if stack is full (in case of array):

• Diagram:

Top ↓

40
30
20
10

← Last inserted (first removed)

← First inserted

Push (50) $\rightarrow$ Added at top

Pop () $\rightarrow$ Removes 40 first.

• Applications of stack:

1] Expression Evaluation:

- Used to evaluate mathematical expression like:
- Infix $\rightarrow (A+B)$
- Postfix $\rightarrow AB^+$
- Prefix $\rightarrow +AB$
- compilers use stacks for expression processing.

2] Function calls (Recursion):

- stack is used in function calling and recursion.
- Each function call is stored in stack memory.
- When function call completes, it is popped.
- Example: Factorial using recursion.

3] Parenthesis checking:

- Used to check balanced parentheses:
 $\{, \}, [,], (,)$
- Helps in syntax checking in programs.

4] Undo / Redo operations:

- Used in applications like:
 - Text editors (ctrl + z / ctrl + y)
- Recent actions are stored in stack.

5] Back tracking Algorithm:

- Used in problems like:
 - Maze solving.

• Sudoku

- Helps to go back previous steps.

6] Reversing Data:

- Stack is used to reverse:

• strings

• Numbers

- Example: Reverse "ABC" $\rightarrow$ "CBA"

7] Browser History:

- Browser use stack for:

• Back and forward navigation.

Q.2 Explain different operations on stack using Array.

$\rightarrow$ A stack can be implemented using an array, where elements are inserted and deleted from only one end called Top.

- It follows the LIFO (Last In First out) principle.

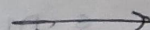
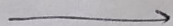
• Representation of Stack using Array:

- A stack is declared as: `int stack [MAX];`
- A variable top is used to track the top element.
- Initially: `top = -1` (stack is empty).

• Basic operations on stack:

1] Push (Insertion):

- Adds an element to the top of the stack.



- Algorithm:

- 1] check if $top == MAX - 1$
 → stack overflow (cannot insert)

- 2] Else
 → $top = top + 1$
 → $stack[top] = element$

- Example:

Push(10), Push(20), Push(30)

stack : [10, 20, 30]

Top → 30

- 2] Pop (Deletion):

- Remove the top element from the stack.

- Algorithm:

- 1] check if $top == -1$
 → stack underflow (empty stack)

- 2] Else
 → $Element = stack[top]$
 → $top = top - 1$

- Example:

Before pop : [10, 20, 30]

After pop : [10, 20]

- 3] Peek (Top):

- Display the top element without removing it.

• Algorithm:

1] IF $top == -1$
→ stack is empty

2] Else
→ Return stack [top]

• Example:

Stack : [10, 20, 30]

peek : → 30

4] isEmpty operation:

- check whether the stack is empty.

• condition:

if ($top == -1$)
⇒ stack is Empty.

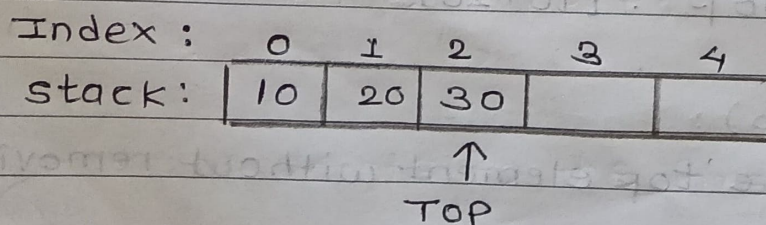
5] isFull operation:

- checks whether the stack is full.

• condition:

if ($top == MAX - 1$)
⇒ stack is Full.

• Diagram of stack (Array of Implementation):



- c program for stack operation:

```
#include <stdio.h>
#define MAX 5
int stack [MAX];
int top : -1;
void push (int x)
{
    if (top == MAX-1)
        printf ("stack overflow \n");
    else
    {
        top ++;
        stack [top]=x;
    }
}
void pop ()
{
    if (top == -1)
        printf ("stack underflow \n");
    else
    {
        printf ("deleted element : %.d \n", stack [top]);
        top --;
    }
}
void peek ()
{
    if (top != -1)
        printf ("top element : %.d \n", stack [top]);
}
```



```
int main()
{
    Push (10);
    Push (20);
    Push (30);
    Pop ();
    Peek ();
    return 0;
}
```

Q.3 Explain different operations on stack using Linked List.

→ A stack can also be implemented using a linked list, where each element (node) contains:

- Data
- Pointer to next node.

- In this implementation insertion and deletion are done at the beginning (Top).

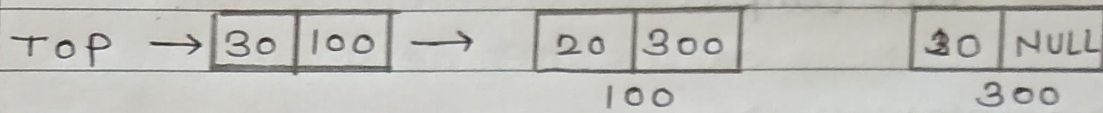
- Representation of stack using Linked list:
Each node:

```
struct node
{
    int data;
    struct node * next;
};
```

- A pointer top points to the first node (top of stack).

- Initially : $top = NULL$ (Stack is Empty).

• Diagram of Stack (Linked List):



Here, 30 is the top element.

• Basic operations on stack:

1] Push operation (Insertion):

- Adds a new element at the beginning (top).

• Algorithm:

- 1) create a new node.
- 2) Insert data into node.
- 3) set new $\rightarrow$ next = top
- 4) Update top = new

• Example:

Push(10), Push(20), Push(30)

Top $\rightarrow$ 30 $\rightarrow$ 20 $\rightarrow$ 10

2] Pop (Deletion) operation:

- Remove the top element.

• Algorithm:

- 1) check if top == NULL
 $\rightarrow$ stack underflow.

2) Else

→ Temp = top

→ top = top → next

→ free temp

- Example:

Before pop : 30 → 20 → 10

After pop : 20 → 10.

3] Peek (Top) operation:

- Displays the top element.

- Algorithm:

1) If top == NULL

→ stack is empty.

2) Else

→ Return top → data.

4] isEmpty operation:

- checks whether stack is empty.

if (top == NULL)

stack is Empty;

- C program for stack using Linked List:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```



```
int data;
struct node * next;
}

struct node * top = NULL;

void push (int x)
{
    struct node * newnode;
    newnode = (struct node *) malloc (size of
                                         (struct node));

    newnode → data = x;
    newnode → next = top;
    top = newnode;
}

void pop()
{
    if (top == NULL)
        printf ("stack underflow \n");
    else
    {
        struct node * temp = top;
        printf ("Deleted element : %d \n", temp → data);
        temp = temp
        top = top → element next;
        free (temp);
    }
}

void peek ()
{
    if (top != NULL)
        printf ("Top element : %d \n", top → data);
}
```



```
int main ()
{
    Push (10);
    Push (20);
    Push (30);
    Pop ();
    Peek ();
    return 0;
}
```

- Advantages of stack using Linked List:
 - Dynamic Size (no fixed limit)
 - No overflow unless memory is full.
 - Efficient memory usage.
- Disadvantages:
 - Extra memory required for pointers.
 - slightly complex implementation.

Q.4 What is queue? Explain different applications of queue.


```
int main ( )  
{  
    Push (10);  
    Push (20);  
    Push (30);  
    Pop ();  
    Peek ();  
    return 0;  
}
```

- Advantages of stack using Linked List:
 - Dynamic Size (no fixed limit)
 - No overflow unless memory is full.
 - Efficient memory usage.
- Disadvantages:
 - Extra memory required for pointers.
 - slightly complex implementation.

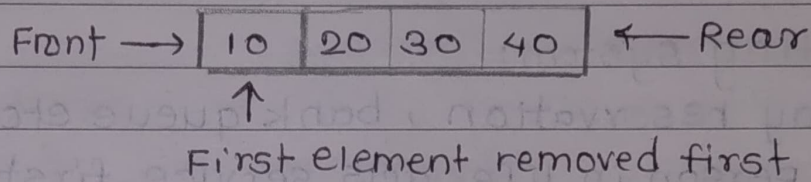
Q.4 What is queue? Explain different application of queue.

→ Queue:

- A queue is a linear data structure that follows the principle of FIFO (First In First out) the element inserted first is removed first.
- It is similar to a line of people waiting at a ticket counter.
- The person who comes first gets served first.

- Basic operations of queue:
 - 1] Enqueue $\rightarrow$ Insert element at the rear.
 - 2) Dequeue $\rightarrow$ Remove element from the front.
 - 3) Peek / front $\rightarrow$ view the front element.
 - 4) isEmpty $\rightarrow$ check if queue is empty.
 - 5) isFull $\rightarrow$ check if queue is Full.

- Diagram of queue:



Enqueue (50) $\rightarrow$ Added at rear

Dequeue () $\rightarrow$ Removes 10 first.

- Applications of queue:

- 1] CPU scheduling (process management):
 - Queue is used in operating system to manage processes.
 - Example: First come first serve (FCFS) scheduling.
- 2] Printer spooling:
 - Print jobs are stored in a queue.
 - First document sent $\rightarrow$ printed first.
- 3] Breadth first search (BFS):
 - Used in graph traversal algorithms.
 - Nodes are visited level by level using.

- 4] Handling Requests in servers:
- Web servers handle multiple user requests using queues.
 - Requests are processed in order of arrival.

- 5] Buffering (data Transfer):
- used in keyboards, networks and IO buffering.
 - Data is stored temporarily before processing.

- 6] Ticketing system:
- Railway reservation, bank queue etc.
 - first person in line gets service first.

- 7] call center system:
- Incoming calls are placed in a queue.
 - calls are attended in order.

- 8] simulation systems:
- Used in traffic management, customer service system.

Q.5 Explain different operations on Linear Queue using Array and Linked List.

- - A linear queue follows the FIFO (First In First out) principle.
- Elements are inserted from the rear and deleted from the front.

• Linear Queue using Array:
Representation:

- Array declaration : `int queue [MAX];`
- Two variables:
 - `front = -1`
 - `rear = -1`

- operations:

1] Enqueue (Insertion):

- Insert element at rear.

- Algorithms:

1] If `rear == Max - 1`

→ Queue overflow.

2) Else

→ • If `front == -1` set `front = 0`

• `rear = rear + 1`

• `queue[rear] = element`

2] Dequeue (Deletion):

- Remove element from front.

- Algorithm:

1) If `front == -1` OR `front > rear`

→ Queue underflow.

2) Else

→ • `element = queue[front]`

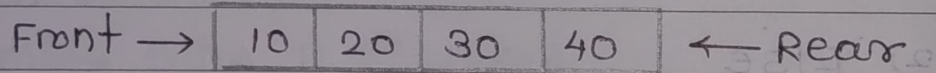
• `front = front + 1`

3] Peek (Front element):
if (front != -1)
return queue [front];

4] isEmpty:
if (front == -1 || front > rear)
Queue is Empty.

5] isFull:
if (rear == MAX-1)
Queue is Full.

• Diagram (Array Queue):



• Linear queue using Linked List:
Representation:

Each node:

```
struct node
{
    int data;
    struct node * next;
};
```

• Two pointers:

- Front
- Rear

• operations:

1] Enqueue (Insertion):

- Insert element at rear.

• Algorithm:

- 1) create new node.
- 2) IF queue is empty.
 - $\text{front} = \text{rear} = \text{new}$
- 3) Else:
 - $\text{rear} \rightarrow \text{next} = \text{new}$

2] Dequeue (Deletion):

- Remove element from front.

• Algorithm:

- 1] IF $\text{front} == \text{NULL}$
 - Queue underflow.

2] Else

- • $\text{temp} = \text{front}$
- $\text{front} = \text{front} \rightarrow \text{next}$
- Free temp.

- 3] IF $\text{front} == \text{NULL} \rightarrow \text{set rear} = \text{NULL}$.

3] Peek:

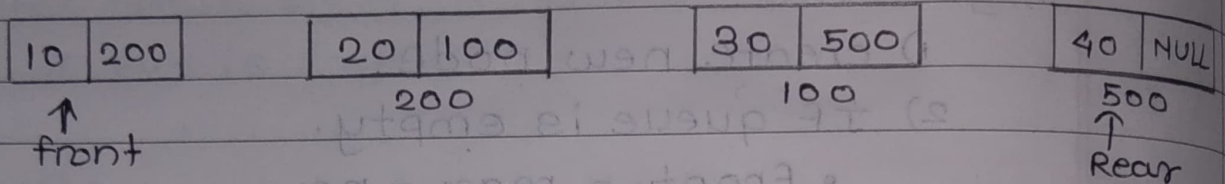
```
if (front != NULL)
    return front → data;
```


4) isEmpty:

if (front == NULL)

Queue is Empty.

• Diagram (Linked List Queue):



• Advantages and Disadvantages:

- simple and easy.
- fixed size overflow problem.

• Linked List Implementation:

- Dynamic size
- Efficient memory usage.
- More complex, extra memory for pointers.

Q.6 Explain circular queue and its application.

→ circular queue:

- A circular queue is a linear data structure in which the last position is connected back to the first position, forming a circle.
- It is an improved version of a linear queue that avoids wastage of space.
- In a normal queue once the rear reaches the end no more insertions are possible even if there is space at the front.
- circular queue solves this by reusing empty space.

- key concept:
 - Positions are reused using module operations.
 - When rear reaches the end, it moves to the beginning.

- conditions:

- Queue is Full:

$$(rear + 1) \% MAX == front$$

- Queue is Empty:

$$front == -1$$

- operations on circular queue:

- 1] Enqueue (Insertion):

- Insert element at rear.

- Algorithm:

1) IF queue is full
→ overflow.

2) IF queue is Empty.
→ $front = rear = 0$

3) ELSE

→ $rear = (rear + 1) \% MAX$

4) Insert element at queue [rear]:

- 2] Dequeue (deletion):

- Remove element from front.

8

10

• Algorithm:

1) If queue is empty.
→ underflow.

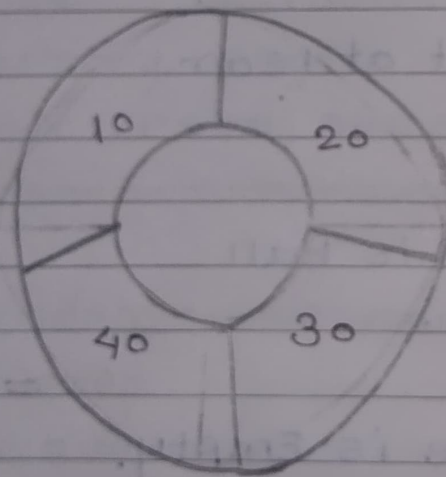
2) If only one element.
→ $\text{front} = \text{rear} = -1$

3) Else
→ $\text{front} = (\text{front} + 1) \% \text{MAX}$

• Peek:

- Return $\text{queue}[\text{front}]$.

• Diagram of circular queue:



Front = 10

Rear = 40

• Advantages of circular queue:

- efficient memory utilization
- No shifting of elements required
- Faster than linear queue in many cases.

- Applications of circular queue:
 - 1) CPU scheduling (Round Robin):
 - Each process gets equal time.
 - After execution, process goes to rear again.
 - 2) Memory management:
 - Used in buffering and memory allocation.
 - 3) Traffic control system:
 - Traffic lights use circular queue logic.
 - 4) Multimedia Applications:
 - Audio / video streaming uses circular buffers.
 - 5) Keyboard buffer:
 - Stores keystrokes in circular manner.
 - 6) Printer spooling:
 - Print jobs processed continuously in loop.

Q.7 Write a program to implement different operation on circular queue using array.

→ - A circular queue uses an array with two pointers.

- Front → Points to first element
- Rear → Points to last element.

It follows FIFO and uses modulo (%) for circular movement.

• Program:

```
#include <stdio.h>
#define MAX 5
int queue [MAX];
int front = -1, rear = -1;
void enqueue (int x)
{
    if ((rear + 1) % MAX == front)
    {
        printf ("queue overflow \n");
    }
    else if (front == -1 & rear == -1)
    {
        front = rear = 0;
        queue [rear] = x;
    }
    else
    {
        rear = (rear + 1) % MAX;
        queue [rear] = x;
    }
}
void dequeue ()
{
    if (front == -1)
    {
        printf ("queue underflow \n");
    }
}
```


13

```

    }
    else if (front == rear)
    {
        printf ("Deleted element: %d \n", queue
                [front]);
        front = rear = -1;
    }
    else
    {
        printf ("Deleted element: %d \n", queue[front]);
        front = (front + 1) % MAX;
    }
}

void display ()
{
    if (front == -1)
    {
        printf ("Queue is Empty \n");
    }
    else
    {
        int i = front;
        printf ("Queue elements :");
        while (i != rear)
        {
            printf ("%d", queue[i]);
            i = (i + 1) % MAX;
        }
        printf ("%d \n", queue[rear]);
    }
}

```

```

void peek ()
{
    if (front == -1)
        printf ("Queue is Empty \n");
    else
        printf ("Front element : %d \n", queue[front]);
}

int main ()
{
    enqueue (10);
    enqueue (20);
    enqueue (30);
    enqueue (40);
    display ();
    dequeue ();
    dequeue ();
    display ();
    enqueue (50);
    enqueue (60);
    display ();
    peek ();
    return 0;
}

```

• Explanation of operations:

- 1) Enqueue → Adds element using $(\text{rear} + 1) \% \text{MAX}$
- 2) Dequeue → Remove element using $(\text{front} + 1) \% \text{MAX}$
- 3) Display → Traverses circularly from front to rear
- 4) Peek → Shows front element.

Q.8 Write a program to implement different operations on circular queue using linked list.

→ In a circular queue using Linked list, the Last node points back to the first node, forming a circle.

- We use two pointers:

- front → first node
- rear → last node

• Program:

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node * next;
};
struct node * front = NULL, * rear = NULL;
void enqueue (int x)
{
    struct node * newnode;
    newnode = (struct node *) malloc (sizeof (struct
node));
    newnode → data = x;
    if (front == NULL)
    {
        front = rear = newnode;
        rear → next = front;
    }
    else
    {
```

15

16

```
rear → next = newnode;
```

```
rear = newnode;
```

```
rear → next = front;
```

```
}
```

```
}
```

```
void dequeue ()
```

```
{
```

```
    if (front == NULL)
```

```
{
```

```
    printf ("Queue Underflow \n");
```

```
}
```

```
    elseif (front == rear)
```

```
{
```

```
    printf ("Deleted element : %d \n", front → data);
```

```
    free (front);
```

```
    front = rear = NULL;
```

```
}
```

```
    else
```

```
{
```

```
        struct node * temp = front;
```

```
        printf ("Deleted element : %d \n", temp → data);
```

```
        front = front → next;
```

```
        rear → next = front;
```

```
        free(temp);
```

```
}
```

```
}
```

```
void display ()
```

```
{
```

```
    if (front == NULL)
```

```
{
```

```
    printf ("Queue is Empty \n");
```

```
}
```



```
    else
    {
        struct node * temp = front;
        Printf ("queue elements: ");
        do {
            Print f ("%d ", temp → data);
            temp = temp → next;
        }
        While (temp != front);
        Printf ("\n");
    }
}

void peek ()
{
    if (front == NULL)
        Print ("queue is Empty \n");
    else
        Printf ("front element : %d \n"; front → data);
}

int main ()
{
    enqueue (10);
    enqueue (20);
    enqueue (30);
    enqueue (40);
    display ();
    dequeue ();
    dequeue ();
    display ();
    enqueue (50);
    display ();
    peek ();
    return 0;
}
```

17

Q.9 Explain how postfix expression is evaluated using stack with example.

→ A postfix expression is an in which the operator comes after the operands.

• Example:

• Infix : $(A+B)$

• Postfix : AB^+

18

• concept of Evaluating using stack:

A stack is used to evaluate postfix expressions efficiently.

• Algorithm:

- 1) Scan the postfix expression from left to right.
- 2) If the symbol is an operand, push it into the stack.
- 3) If the symbol is an operation:
 - Pop two elements from the stack.
 - Perform operation $\rightarrow$ operand 1 operator operand 2.
 - Push the result back to the stack.
- 4) Repeat until expression ends.
- 5) Final result will be at the top of the stack.

• Example:

Evaluate postfix Expression

$23^* 54^+ + 9^-$

→ → →

• Step by - step Evaluation:

step	Symbol	Action	Stack
1	2		
2	3	Push	2
3	*	Push	2, 3
4	5	$2 \times 3 = 6 \rightarrow$ Push	6
5	4	Push	6, 5
6	*	Push	6, 5, 4
7	+	$5 \times 4 = 20 \rightarrow$ Push	6, 20
8	9	$6 + 20 = 26 \rightarrow$ Push	26
9	-	Push	26, 9
		$26 - 9 = 17 \rightarrow$ Push	17

Final Result:

Result = 17

17

• step Example:

Push 2 $\rightarrow$ [2]

Push 3 $\rightarrow$ [2, 3]

* $\rightarrow$ [6]

Push 5 $\rightarrow$ [6, 5]

Push 4 $\rightarrow$ [6, 5, 4]

* $\rightarrow$ [6, 20]

+ $\rightarrow$ [26]

Push 9 $\rightarrow$ [26, 9]

- $\rightarrow$ [17]

• Advantages:-

- ① No need for parantheses.
- ② Easy to evaluate using stack.
- ③ Faster computation for computers.